CHIABERGE Christopher / BOULANGER Louis / IJJA Ziad

ALMA - TP Noté Deep Learning

Les codes modifiés sont présents dans le dossier joint, mais aussi directement sur le pdf.

EXERCICE 1 :

Question 1 : L'apprentissage profond utilisé dans cet algorithme consiste en :

- 1 . La première étape consiste en la préparation des données.

Avant l'entraînement, les images sont préparées afin de faciliter l'apprentissage du modèle. Les valeurs des pixels sont normalisées entre 0 et 1. Cette opération permet notamment d'obtenir des valeurs plus faciles à exploiter par le réseau. L'idée est d'uniformiser les données, car certaines peuvent être erronées, ou manquantes, ou de mauvaise taille etc. En l'occurrence, ici on resize les images.

- 2 . La seconde étape consiste en la création du modèle.

Ici, le modèle est définis en 7 couches :

- On a 2 couches de convolutions, qui permettent d'extraire des caractéristiques des images, comme les contours, les courbes et les différentes formes qui composent les chiffres. Elles utilisent des filtres qui parcourent l'image afin de détecter ces caractéristiques.
- Les fonctions d'activation (ReLu en l'occurrence) introduisent de la non-linéarité dans le réseau. Cela permet au modèle d'apprendre des relations plus complexes que celles qu'il pourrait représenter avec de simples opérations linéaires.
- Les couches de MaxPooling réduisent la taille des cartes de caractéristiques en conservant les valeurs maximales dans des zones locales. Elles diminuent ainsi le nombre de valeurs à traiter tout en conservant une partie des informations importantes.
- La couche Flatten transforme ensuite les cartes de caractéristiques en un vecteur.
- La couche Dropout désactive aléatoirement certains neurones pendant l'entraînement. Cette technique permet de limiter l'overfitting.
- Enfin, la dernière couche Dense utilise une fonction d'activation Softmax pour produire une probabilité pour chacune des dix classes possibles.

- 3 . La troisième étape consiste en l'entraînement et l'évaluation du modèle.

Pendant l’entraînement, le modèle effectue des prédictions à partir des images d’entrée. Ces prédictions sont comparées aux étiquettes attendues à l’aide d’une fonction de perte, qui mesure l’écart entre les résultats obtenus et les résultats attendus. La backpropagation permet ensuite de calculer les gradients de cette fonction de perte par rapport aux paramètres du réseau. L’optimiseur utilise ces gradients pour mettre à jour les poids et les biais du modèle afin de réduire progressivement l’erreur. Ce processus est répété sur plusieurs époques. Une époque correspondant à un passage complet sur les données d’entraînement.

Une fois l’entraînement terminé, le modèle est évalué sur les données de test afin de mesurer sa capacité à reconnaître des images qu’il n’a pas utilisées pour ajuster ses paramètres. La précision indique la proportion d’images correctement classées.

Le modèle est finalement évalué et montre une précision >99 %.

```
Test loss: 0.02499214932322502

Test accuracy: 0.9919000267982483
```

Question 2 :

La préparation des données consiste à mettre les images et leurs étiquettes dans un format adapté au modèle, afin de faciliter son apprentissage.

Tout d'abord, le jeu de données MNIST est chargé à l'aide de Keras. Il contient 70 000 images de chiffres manuscrits en niveaux de gris, réparties entre 60 000 images d'entraînement et 10 000 images de test. Chaque image possède une taille de 28 × 28 pixels.

Ensuite, les valeurs des pixels sont normalisées entre 0 et 1 :

```
x_train = x_train.astype("float32") / 255

x_test = x_test.astype("float32") / 255
```

De base chaque pixel possède une valeur comprise entre 0 et 255 correspondant à son intensité. La conversion en nombres flottants et la division par 255 permet d'obtenir des valeurs comprises entre 0 et 1. Cela facilite généralement l'apprentissage en fournissant au réseau des entrées sur une échelle homogène.

Les images sont ensuite mises sous la forme attendue par le CNN :

```
x_train = np.expand_dims(x_train, -1)

x_test = np.expand_dims(x_test, -1)
```

Cette opération ajoute une dimension correspondant au canal de couleur. Comme les images sont en niveaux de gris, elles possèdent un seul canal. Leur forme passe donc de (28, 28) à (28, 28, 1).

Enfin, les étiquettes sont converties en matrices binaires à l'aide de la fonction to_categorical:

```
y_train = keras.utils.to_categorical(y_train, num_classes)

y_test = keras.utils.to_categorical(y_test, num_classes)
```

Chaque étiquette correspond à un entier compris entre 0 et 9. Après cette conversion, elle est représentée par un vecteur de dix valeurs. Par exemple, le chiffre 3 est représenté par le vecteur [0, 0, 0, 1, 0, 0, 0, 0, 0, 0] (la valeur 1 indique la classe correcte). Cette représentation est adaptée à la fonction de perte categorical_crossentropy qui est utilisée pendant l'entraînement pour comparer les prédictions du modèle aux classes attendues.

Après avoir fait ces opérations, les images et leurs étiquettes sont dans un le bon format qui est compatible avec le modèle et peuvent être utilisées pour son entraînement et son évaluation.

Question 3 :

Le modèle construit est un CNN. Son objectif est d'extraire progressivement les caractéristiques d'une image puis de les utiliser pour déterminer à quel chiffre elle correspond. Le modèle reçoit en entrée une image de forme (28,28,1).

La première couche est une couche de convolution Conv2D utilisant 32 filtres de taille 3×3 et ReLU. Chaque filtre parcourt l'image afin de détecter des caractéristiques locales (ex: contours ou courbes). Avec les paramètres par défaut de Keras, la convolution n'ajoute pas de padding : la taille spatiale passe donc de 28×28 à 26×26. Comme 32 filtres sont utilisés, la sortie possède la forme (26,26,32).

Une couche MaxPooling2D de taille 2×2 est ensuite appliquée. Elle conserve la valeur maximale dans chaque zone de 2×2 pixels. Cela réduit de 26×26 à 13x13. Cette opération diminue le nombre de valeurs à traiter et rend les caractéristiques moins sensibles à de petits déplacements dans l'image.

Le réseau utilise ensuite une deuxième couche de convolution, avec 64 filtres de taille 3×3. Cette couche apprend des caractéristiques plus complexes à partir de celles extraites précédemment. La taille passe de 13×13 à 11×11. Donc la sortie possède la forme (11,11,64). Une deuxième couche de MaxPooling réduit ensuite cette taille à (5,5,64).

La couche Flatten transforme en un vecteur de 1 600 valeurs (5×5×64=1600). Cette transformation permet de transmettre les caractéristiques extraites à la couche de classification.

Une couche Dropout avec un taux de 0,5 est ensuite utilisée pour limiter le surapprentissage. Pendant l'entraînement, elle désactive aléatoirement 50 % des unités de son entrée à chaque passage.

Enfin, la couche Dense comporte dix neurones (10 classes) La fonction d'activation Softmax transforme leurs scores en probabilités dont la somme vaut 1. Le chiffre prédit est celui qui possède la probabilité la plus élevée.

L'ordre des couches permet donc de passer d'une représentation spatiale de l'image à des caractéristiques de plus en plus élaborées puis à une classification. Les convolutions extraient les caractéristiques, le MaxPooling réduit les dimensions spatiales, Flatten prépare les données pour la classification et la couche Dense détermine la classe finale. Dropout aide à améliorer la capacité de généralisation du modèle.

Question 4 :

Pour la modifications d'hyper paramètres :

1/ kernel_size(3,3) à kernel_size(5,5) pour toutes les convolutions.

En augmentant la taille du kernel, le modèle ne va plus faire ses convolutions sur une zone de 3x3 pixels, mais sur une zone de 5x5 pixels. Cela fait que le modèle a alors une vue plus “reculée”, et ne se concentre moins sur les fins détails (courbure du chiffre etc). Pour des inputs de petites tailles telles que MNIST, la différence est assez minime, et le modèle ne diffère pas trop sur la précision, en réalité il semble même légèrement s’améliorer (0.9919 à 0.9937) mais cela peut aussi être dû au fait que les entraînements et tests ne soient pas fait sur une même seed. Par ailleurs, le nombre de paramètres du modèle augmente (62 346 contre 34 826).

```
test d’accuracy : 0.9937999844551086

test de loss : 0.0186168160289526
```

Code modifié :

```
model = keras.Sequential(
        [
              keras.Input(shape=input_shape),
              layers.Conv2D(32, kernel_size=(5, 5), activation="relu"),
              layers.MaxPooling2D(pool_size=(2, 2)),
              layers.Conv2D(64, kernel_size=(5, 5), activation="relu"),
              layers.MaxPooling2D(pool_size=(2, 2)),
              layers.Flatten(),
             layers.Dropout(0.5),
```

```
            layers.Dense(num_classes, activation="softmax"),
        ]
)
model.summary()
```

2/ Réduction du nombre d’epochs d'entraînement (15 à 5) :

En réduisant le nombre d’epochs, le modèle aura fini son apprentissage plus rapidement, mais il n’aura peut être pas suffisamment mis à jour ses poids, résultant d’une accuracy pas optimal, et d’une loss n’ayant pas atteint sa valeure minimale.

```
test d’accuracy : 0.9879999756813049

test de loss : 0.0367610827088356
```

Comme prévu, la précision est inférieure (différence très négligeable pour cet exemple), et la loss est plus grande, démontrant bien que le modèle a “moins appris” qu’auparavant. En réalité, pour être sûr que le modèle ait suffisamment appris ou non, il faudrait comparer les courbes d’accuracy et de loss des 2 modèles, ici nous nous basons uniquement sur la loss

Code modifié :

```
batch_size = 128
epochs = 5

model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])

model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs, validation_split=0.1)
```

.

3/ Modification du modèle :

La première modification du modèle consiste en une réduction du modèle initial. Nous retirons les 2 couches successives suivantes :

```
layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),

layers.MaxPooling2D(pool_size=(2, 2)),
```

Concrètement, la suppression de la couche de convolution réduit la capacité du réseau à extraire des caractéristiques à partir des images. Le modèle dispose alors de moins de filtres pour apprendre des motifs complexes :

```
Test loss: 0.04886912181973457
Test accuracy: 0.9847000241279602
```

Le modèle est alors moins complexe, et logiquement moins précis.

Code modifié :

```
model = keras.Sequential(
        [
                keras.Input(shape=input_shape),
                layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
                layers.MaxPooling2D(pool_size=(2, 2)),
                layers.Flatten(),
               layers.Dropout(0.5),
                layers.Dense(num_classes, activation="softmax"),
        ]
)
model.summary()
```

Une autre modification du modèle peut être dans l'autre sens, cette fois-ci, nous ajoutons une couche convolutive de plus au modèle.

Pour ajouter une nouvelle couche de convolution, il suffit de l'ajouter à la suite du MaxPooling2D. Pour déterminer le nombre de filtres, étant donné que le modèle et les inputs ne sont pas très complexes, nous gardons 64 filtres, mais il aurait aussi été possible de mettre 128 filtres ou autre, néanmoins plus de filtres équivaut à une plus grande complexité, sans pour autant être plus précis.

```
Test loss: 0.021849343553185463
Test accuracy: 0.9927999973297119
```

Le modèle est alors légèrement plus précis que sa version initiale

Code modifié :

```
model = keras.Sequential(
        [
                keras.Input(shape=input_shape),
                layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
                layers.MaxPooling2D(pool_size=(2, 2)),
                layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),
                layers.MaxPooling2D(pool_size=(2, 2)),
                layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),
                layers.Flatten(),
               layers.Dropout(0.5),
                layers.Dense(num_classes, activation="softmax"),
        ]
)
model.summary()
```

EXERCICE 2 :

Question 1:

Les RNN sont des modèles conçus pour traiter des données séquentielles (texte, parole, séries temporelles) en conservant une mémoire interne des entrées précédentes via des connexions récurrentes.
Dans le cadre de ce model, il est utilisé pour reconnaître les captcha en le lisant avec deux couches Bidirectional (lis dans les deux sens) LSTM malgré que les lettres soient modifiées.

Question 2:

Preprocessing :

Les caractères sont transformés en integers pour être traité par le model. Chaque image est lue, convertie en niveaux de gris, normalisée entre 0 et 1, redimensionnée à des dimensions fixes, puis transposée. Enfin le texte associé à l'image est découpé en caractères et converti en séquence que le RNN pourra lire.

Fonction CTL :

Des fonctions CTL sont définies avant le build du modèle. La fonction `ctc_label_dense_to_sparse()` convertit les étiquettes en format adapté au calcul de la perte. La fonction `ctc_batch_cost()` calcule l'erreur entre les prédictions du réseau et les textes attendus, sans besoin d'alignement précis entre les caractères et sa position dans l'image. Enfin, la couche `CTCLayer` intègre cette perte au modèle grâce à `add_loss()`. Cette méthode permet au réseau d'apprendre à reconnaître les séquences de caractères.

Build model :

Le model lit un captcha comme une séquence. Les images en niveaux de gris passent par deux couches Conv2D et MaxPooling qui extraient les features, donc utilise aussi un CNN. La sortie est reshape en une séquence de 50 colonnes, chacune décrite par 768 features. Deux LSTM bidirectionnels lisent cette séquence dans les deux sens, et une couche Dense softmax donne pour chaque colonne une probabilité sur les caractères possibles. Comme il y a plus de colonnes que de lettres, le modèle est entraîné avec la loss CTC, qui aligne automatiquement la sortie sur le texte. À la prédiction, le décodage CTC garde le caractère le plus probable par colonne, fusionne les répétitions et supprime les blancs.

Question 3 :

Modifications structurelles :

Supprimer les convolutions :

Le réseau reçoit directement les pixels de l'image sous forme de séquence, sans extraction préalable des caractéristiques par CNN.

Hypothèse : la suppression des couches convolutives risque de diminuer les performances du modèle, car le RNN devra apprendre directement à partir des pixels sans bénéficier d'une extraction préalable des caractéristiques visuelles. Le temps d'entraînement pourrait également augmenter.

Changements de paramètres et hyperparamètres :

Plus grand batch_size (32/64) pour gagner du temps par epochs

Hypothèse : l'augmentation du batch_size à 32 ou 64 pourrait réduire le temps d'entraînement par époque en traitant plus d'images simultanément. Cependant, elle augmentera la consommation de mémoire et pourrait modifier la convergence du modèle ainsi que ses performances de reconnaissance.

Modifier les fonctions d'activations :

Remplacer `relu` par `tanh` dans les couches convolutifs et la couche `Dense` intermédiaire.

Hypothèse : le remplacement de relu par tanh dans les couches concernées modifiera la distribution des activations et la propagation des gradients. Cela pourrait influencer la vitesse de convergence et la précision de reconnaissance, positivement ou négativement selon les résultats expérimentaux.

Malheureusement, le programme présent sur keras ne s’exécute pas, nous ne pouvons alors qu'émettre des suppositions sur ce qui pourrait changer pour le modèle.